

دانشگاه صنعتی خواجه نصیرالدین طوسی دانشکده مهندسی برق

درس مبانی سیستم های هوشمند

مینی پروژه ۴

امیرحسین رضانی ۴۰۰۰۵۷۲۳ – مهسان قلمی ۴۰۰۰۸۱۵۳

لینک های گیت هاب

<https://github.com/MahsanGhalami/Fundamentals-of-Intelligent-Systems-4041>



پرسش یک

آ) بارگذاری داده‌ها و پیش‌پردازش اولیه

```
# Load train and test datasets
train_data = pd.read_csv("loan_train.csv")
test_data = pd.read_csv("loan_test.csv")
train_data.info()
```

```
train_data.info()
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 614 entries, 0 to 613
Data columns (total 12 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Gender                601 non-null   object
1   Married               611 non-null   object
2   Dependents            599 non-null   object
3   Education              614 non-null   object
4   Self_Employed         582 non-null   object
5   Applicant_Income      614 non-null   int64
6   Coapplicant_Income    614 non-null   float64
7   Loan_Amount           614 non-null   int64
8   Term                  600 non-null   float64
9   Credit_History         564 non-null   float64
10  Area                  614 non-null   object
11  Status                 614 non-null   object
dtypes: float64(3), int64(2), object(7)
memory usage: 57.7+ KB
```

```
test_data.info()
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 367 entries, 0 to 366
Data columns (total 11 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Gender                356 non-null   object
1   Married               367 non-null   object
2   Dependents            357 non-null   object
3   Education              367 non-null   object
4   Self_Employed         344 non-null   object
5   Applicant_Income      367 non-null   int64
6   Coapplicant_Income    367 non-null   int64
7   Loan_Amount           367 non-null   int64
8   Term                  361 non-null   float64
9   Credit_History         338 non-null   float64
10  Area                  367 non-null   object
dtypes: float64(2), int64(3), object(6)
memory usage: 31.7+ KB
```

```
# Separate features and target variable
X = train_data.drop("Status", axis=1)
y = train_data["Status"]
```



```
# Fix Dependents like '3+' -> 3 (makes it numeric)
for df in (X, test_data):
    if "Dependents" in df.columns:
        df["Dependents"] = df["Dependents"].replace({"3+": "3"})

# Handle missing values in both datasets
for col in X.columns:
    if X[col].dtype == "object":
        X[col] = X[col].fillna(X[col].mode()[0])
        test_data[col] = test_data[col].fillna(test_data[col].mode()[0])
    else:
        X[col] = X[col].fillna(X[col].median())
        test_data[col] = test_data[col].fillna(test_data[col].median())
```

- فایل train شامل ۶۱۴ رکورد و ۱۲ ستون است (۱۱ ویژگی ورودی و برچسب Status برای تایید/رد وام) و در ستون‌هایی مثل Gender, Dependents, Self_Employed, Credit_History, Term مقدار گمشده وجود دارد. فایل test دارای ۳۶۷ رکورد و ۱۱ ستون است و ستون Status را ندارد؛ بنابراین فقط برای پیش‌بینی نهایی استفاده می‌شود.
- برای یکدست‌سازی داده‌ها، مقدار متنی "۳+" در ستون Dependents به "۳" تبدیل شد تا در ادامه امکان تبدیل آن به مقدار عددی و کدگذاری بهتر فراهم شود.
- در گام بعد، مقادیر گمشده در هر دو مجموعه‌ی فایل‌های train و test مدیریت شد: برای ستون‌های اسمی (نوع object) مقدار پرتکرار (mode) و برای ستون‌های عددی مقدار میانه (median) جایگزین NaN شد. این روش بدون حذف ردیف‌ها، داده را کامل و آماده‌ی ورود به مدل می‌کند.
- در نهایت، با توجه به نبود برچسب در فایل test، کل فرآیند یادگیری شامل Feature Selection و آموزش مدل فقط روی داده‌ی train انجام می‌شود و پس از آموزش، مدل در مرحله‌ی پایانی روی فایل test اجرا شده و خروجی پیش‌بینی وضعیت وام برای نمونه‌های test تولید خواهد شد.

```
# Encode categorical features
categorical_cols = X.select_dtypes(include=["object"]).columns
encoder = LabelEncoder()

for col in categorical_cols:
    encoder.fit(X[col]) # fit only on training data
    X[col] = encoder.transform(X[col])
    test_data[col] = encoder.transform(test_data[col])

# Encode target Status
y_le = LabelEncoder()
y = y_le.fit_transform(y.astype(str)) # e.g., N->0, Y->1
```

- در این مرحله ستون‌های اسمی (نوع object) شناسایی شدند و با LabelEncoder به عدد تبدیل شدند تا مدل بتواند با آن‌ها کار کند. برای هر ستون، encoder فقط روی داده‌ی فایل train، fit شد



و سپس همان نگاشت روی داده ی فایل train و داده ی فایل test, transform شد تا کدگذاری هر دو مجموعه یکسان باشد و نشت اطلاعات رخ ندهد.

- در نهایت، برچسب خروجی Status نیز با یک LabelEncoder جداگانه به مقدار عددی تبدیل شد ($Y=1$ و $N=0$) تا برای آموزش مدل قابل استفاده باشد.

```
# Split training data into train and validation sets
X_train, X_val, y_train, y_val = train_test_split(X, y, test_size=0.3,
random_state=53)
```

- در این مرحله، داده‌های آموزش برای ارزیابی بهتر مدل به دو بخش تقسیم شد: مجموعه آموزش (Train) و مجموعه اعتبارسنجی (Validation). ۷۰٪ داده‌ها برای آموزش مدل و ۳۰٪ برای بررسی عملکرد و تنظیمات مدل کنار گذاشته شد.

تعریف مدل پایه و تابع ارزیابی (Random Forest + Metrics)

```
SEED = 53
ALPHA = 0.5 # trade-off between accuracy and number of selected features

def make_rf(n_estimator, seed=53):
    return
    RandomForestClassifier(n_estimators=n_estimator, random_state=seed, n_jobs=-
1, max_depth=10)

def evaluate_model(model, Xtr, ytr, Xte, yte, feature_mask=None):
    """Train on (Xtr,ytr) and evaluate on (Xte,yte).
    feature_mask: boolean mask over columns (length = n_features), or None for
    all features.
    """
    if feature_mask is not None:
        Xtr = Xtr.iloc[:, feature_mask]
        Xte = Xte.iloc[:, feature_mask]
    model.fit(Xtr, ytr)
    pred = model.predict(Xte)
    out = {
        "accuracy": float(accuracy_score(yte, pred)),
        "precision": float(precision_score(yte, pred, zero_division=0)),
        "recall": float(recall_score(yte, pred, zero_division=0)),
        "f1": float(f1_score(yte, pred, zero_division=0)),
        "confusion_matrix": confusion_matrix(yte, pred),
    }
    return out
```

- در این بخش، پارامتر ALPHA=0.5 به عنوان ضریب توازن بین عملکرد مدل (دقت) و تعداد ویژگی‌های انتخاب شده در تابع برازندگی در نظر گرفته شد. دلیل انتخاب مقدار ۰.۵ این است تا هیچ یک از دو هدف



- غالب نشود و یک تعادل منطقی برقرار گردد؛ یعنی نه صرفاً با نگه داشتن ویژگی‌های زیاد دقت را بیشینه کنیم و نه آن قدر ویژگی‌ها را حذف کنیم که عملکرد مدل افت محسوسی پیدا کند.
- به طور کلی اگر α بزرگ‌تر (نزدیک ۱) باشد، تمرکز بیشتر روی افزایش دقت است و معمولاً ویژگی‌های بیشتری انتخاب می‌شود؛ و اگر α کوچک‌تر (نزدیک ۰) باشد، اولویت با کمینه‌سازی تعداد ویژگی‌ها است و ممکن است دقت تا حدی کاهش یابد.
 - سپس تابع `make_rf` مدل `RandomForestClassifier` با تعداد درخت‌های قابل تنظیم، عمق بیشینه ۱۰ برای کنترل پیچیدگی و زمان محاسبات، و استفاده از تمام هسته‌های پردازنده (`n_jobs=-1`) ایجاد می‌کند.
 - در ادامه، تابع `evaluate_model` در صورت دریافت `feature_mask` فقط زیرمجموعه ویژگی‌های انتخاب‌شده را نگه می‌دارد، مدل را روی داده‌ی آموزش `fit` کرده و روی داده‌ی ارزیابی پیش‌بینی می‌گیرد و معیارهای `Accuracy`، `Precision`، `Recall`، `F1` و ماتریس درهم‌ریختگی (`Confusion Matrix`) را گزارش می‌دهد.

ب) الگوریتم Particle Swarm Optimization (PSO)

```
n_features = X_train.shape[1]
```

Continuous PSO

```
from pyswarms.single import GlobalBestPSO

options_cont = {"c1": 0.5, "c2": 0.3, "w": 0.9}

pso_cont = GlobalBestPSO(
    n_particles=50,
    dimensions=n_features,
    options=options_cont
)

def pso_cont_cost(particles):
    costs = []
    for particle in particles:
        mask = particle > 0.5 # threshold

        if mask.sum() == 0:
            costs.append(1.0)
            continue

    rf = make_rf(1, SEED)
```



```

metrics = evaluate_model(rf, X_train, y_train, X_val, y_val,
feature_mask=mask)

acc = metrics["accuracy"]
feat_ratio = mask.sum() / n_features

J = ALPHA * (1 - acc) + (1 - ALPHA) * feat_ratio
costs.append(J)

return np.array(costs)

best_cost_cont, best_pos_cont = pso_cont.optimize(pso_cont_cost, iters=100,
verbose=False)

pso_cont_mask = best_pos_cont > 0.5
pso_cont_features = X.columns[pso_cont_mask].tolist()

print("PSO (continuous) best cost:", best_cost_cont)
print("PSO (continuous) selected features:", int(pso_cont_mask.sum()))
print("PSO selected features:", pso_cont_features)
PSO (continuous) best cost: 0.15085995085995085
PSO (continuous) selected features: 1
PSO selected features: ['Credit_History']

```

- در این بخش، یک نسخه PSO پیوسته (Continuous PSO) با استفاده از کلاس GlobalBestPSO از کتابخانه pyswarms برای انتخاب ویژگی پیاده‌سازی شد. تعداد ابعاد هر ذره برابر با تعداد کل ویژگی‌ها (n_features) در نظر گرفته شد؛ بنابراین هر ذره یک بردار پیوسته به طول ویژگی‌ها تولید می‌کند. برای تبدیل این بردار به انتخاب ویژگی به صورت باینری، از آستانه‌گذاری استفاده شد؛ به این صورت که مولفه‌های بزرگ‌تر از 0.5 به عنوان ویژگی‌های انتخاب‌شده در نظر گرفته شدند (mask = particle > 0.5). اگر یک ذره هیچ ویژگی‌ای را انتخاب نکند، برای جلوگیری از پاسخ نامعتبر، هزینه‌ی ثابت 1.0 برای آن ثبت شد.
- برای ارزیابی هر ماسک، مدل Random Forest روی داده‌ی آموزش (X_{train} , y_{train}) آموزش داده شد و عملکرد آن روی داده‌ی اعتبارسنجی (X_{val} , y_{val}) سنجیده شد. سپس یک تابع هزینه تعریف شد تا همزمان خطای مدل و تعداد ویژگی‌های انتخاب‌شده را وارد فرآیند تصمیم‌گیری کند:

$$J = \alpha(1 - Acc) + (1 - \alpha) \times feat_ratio$$

که در آن $feat_ratio = \frac{\#selected}{\#total}$ نسبت ویژگی‌های انتخاب‌شده به کل ویژگی‌هاست و α وزن تعادل بین دو هدف را تعیین می‌کند.



- در صورت سوال ترم دوم به شکل $\left(1 - \frac{\#selected}{\#total}\right)$ آمده است، اما در پیاده‌سازی از خود `feat_ratio` استفاده شد؛ چون ما J را کمینه می‌کنیم و با این فرم، هرچه تعداد ویژگی‌ها بیشتر شود، هزینه بزرگ‌تر شده و الگوریتم مستقیماً به سمت زیرمجموعه‌های کوچک‌تر هدایت می‌شود. در عمل نیز وقتی فرم صورت سوال اجرا شد، به دلیل جهت‌دهی ترم دوم، الگوریتم به صورت پایدار به سمت نگه‌داشتن تقریباً همه ۱۱ ویژگی گرایش پیدا می‌کرد؛ به همین دلیل برای اینکه جریمه تعداد ویژگی‌ها واقعاً اثرگذار باشد و انتخاب ویژگی انجام شود، از فرم `feat_ratio` در تابع هزینه استفاده گردید.
- از آنجا که PSO در هر تکرار باید تعداد زیادی مدل را ارزیابی کند، برای کاهش زمان محاسبات مقدار `n_estimators` در `Random Forest` برابر یک قرار داده شد. این ساده‌سازی با توجه به آزمون‌های قبلی انجام شد؛ به طوری که با `n_estimators`های بالاتر نیز الگوی انتخاب ویژگی تغییر محسوسی نداشته و نتیجه‌ی کلی مشابه باقی می‌ماند. در نهایت PSO با ۵۰ ذره و ۱۰۰ تکرار اجرا شد و بهترین پاسخ با هزینه‌ی 0.1509 به دست آمد که تنها یک ویژگی `Credit_History` را انتخاب کرد.
- ❖ انتخاب شدن تنها ویژگی `Credit_History` نشان می‌دهد این متغیر به تنهایی بیشترین قدرت تفکیک بین «تایید» و «رد» وام را داشته و سایر ویژگی‌ها در حضور آن یا اطلاعات افزوده‌ی قابل توجهی به مدل نداده‌اند یا اثرشان نسبت به هزینه‌ی افزایش تعداد ویژگی‌ها (طبق تابع J) کمتر بوده است.
- ❖ از آنجا که تابع هزینه علاوه بر خطا، نسبت تعداد ویژگی‌ها را نیز جریمه می‌کند، PSO به سمت راه‌حلی گرایش پیدا کرده که با حداقل ویژگی‌ها بتواند دقت قابل قبولی تولید کند؛ بنابراین تک‌ویژگی شدن خروجی می‌تواند نتیجه‌ی قدرت بالای `Credit_History` و همزمان فشار بخش کمینه‌سازی ویژگی‌ها در تابع هزینه باشد. به بیان ساده، سیستم اعتبارسنجی سابقه‌ی اعتباری در این داده‌ها نقش بسیار تعیین‌کننده‌ای در تصمیم نهایی داشته و PSO آن را به عنوان موثرترین شاخص شناسایی کرده است.

Binary PSO

```
from pyswarms.discrete.binary import BinaryPSO
```

```
def binary_fitness_func(particles):
```

```
    """
```

```
    particles: (n_particles, n_features) with 0/1 values.
```

```
    """
```

```
    costs = []
```

```
    alpha = ALPHA
```

```
    for particle in particles:
```

```
        selected = particle.astype(bool)
```

```
        # Penalize empty selection
```

```
        if selected.sum() == 0:
```



```
costs.append(1.0)
continue

rf = make_rf(1, SEED)
metrics = evaluate_model(rf, X_train, y_train, X_val, y_val,
feature_mask=selected)

acc = metrics["accuracy"]
feature_ratio = selected.sum() / n_features

# Objective (minimize)
# Common + sensible:
# J = alpha*(1-Acc) + (1-alpha)*(feature_ratio)
cost = alpha * (1 - acc) + (1 - alpha) * feature_ratio
costs.append(cost)

return np.array(costs)

# PSO hyperparams (prompt suggestions + required keys for BinaryPSO)
options = {
    "c1": 0.5,
    "c2": 0.3,
    "w": 0.9,
    "k": 10, # number of neighbors (required by pyswarms BinaryPSO)
    "p": 2   # Minkowski p-norm (usually 1 or 2)
}

optimizer = BinaryPSO(
    n_particles=50,
    dimensions=n_features,
    options=options
)

best_cost_pso, best_pos_pso = optimizer.optimize(binary_fitness_func,
iters=100, verbose=False)

pso_mask = best_pos_pso.astype(bool)
pso_features = X.columns[pso_mask].to_list()
pso_selected_features = pso_features

print("Selected features (BPSO):", pso_features)
print("Selected features:", int(pso_mask.sum()))
print("Best cost:", best_cost_pso)
Selected features (BPSO): ['Credit_History']
Selected features: 1
Best cost: 0.15085995085995085
```

- در ادامه، برای اطمینان از اینکه نتیجه به نوع پیاده‌سازی وابسته نیست، علاوه بر PSO پیوسته، نسخه‌ی PSO باینری (Binary PSO) را هم با کتابخانه pyswarms اجرا کردیم. تفاوت اصلی این روش با



حالت continuous این است که در BPSO هر ذره مستقیماً یک بردار صفر و یک به طول $n_features$ تولید می‌کند (۱ = انتخاب ویژگی، ۰ = حذف ویژگی) و دیگر نیازی به آستانه‌گذاری روی مقادیر پیوسته نیست. در تابع برازندگی نیز اگر هیچ ویژگی انتخاب نشود، برای جلوگیری از پاسخ نامعتبر، هزینه‌ی 1.0 به آن ذره اختصاص داده می‌شود. سپس همان چارچوب ارزیابی قبلی استفاده شد: مدل Random Forest برای کاهش زمان محاسبات با ($n_estimators=1$) روی داده‌ی آموزش fit شد و روی داده‌ی اعتبارسنجی ارزیابی گردید و هزینه با ترکیب خطا و نسبت ویژگی‌های انتخاب‌شده محاسبه شد:

$$J = \alpha(1 - Acc) + (1 - \alpha) \times \text{feature_ratio}$$

- پارامترهای PSO شامل c_1 و c_2 (وزن مولفه‌های شناختی و اجتماعی) و w (اینرسی) تنظیم شدند و برای BinaryPSO نیز پارامترهای k (تعداد همسایه‌ها) و p (نوع نرم فاصله) مطابق نیاز کتابخانه تعیین گردید. الگوریتم با 50 ذره و 100 تکرار اجرا شد و نتیجه‌ی نهایی کاملاً با حالت PSO پیوسته یکسان بود: تنها ویژگی Credit_History انتخاب شد (۱ ویژگی) و مقدار $\text{Best cost} = 0.15086$ به دست آمد. این همسانی خروجی‌ها نشان می‌دهد انتخاب Credit_History یک نتیجه‌ی پایدار و مستقل از نوع نمایش ذره (پیوسته یا باینری) است و در این داده، این ویژگی به‌تنهایی بهترین توازن بین دقت و کمینه‌سازی تعداد ویژگی‌ها را فراهم کرده است.

ج) الگوریتم Genetic Algorithm (GA)

```
from deap import base, creator, tools, algorithms
creator.create("FitnessMin", base.Fitness, weights=(-1.0,))
creator.create("Individual", list, fitness=creator.FitnessMin)
toolbox = base.Toolbox()
toolbox.register("attr_bool", random.randint, 0, 1)
toolbox.register("individual", tools.initRepeat, creator.Individual, toolbox.attr_bool, n=n_features)
toolbox.register("population", tools.initRepeat, list, toolbox.individual)
def ga_fitness_func(individual):
    selected = np.array(individual, dtype=bool)

    if selected.sum() == 0:
        return (1,)

    model = RandomForestClassifier(random_state=42)
    model.fit(X_train.iloc[:, selected], y_train)

    acc = accuracy_score(y_val, model.predict(X_val.iloc[:, selected]))
    alpha = 0.5
    cost = alpha * (1 - acc) + (1 - alpha) * (selected.sum() / n_features)
    return (cost,)
```



```

toolbox.register("evaluate", ga_fitness_func)
toolbox.register("mate", tools.cxTwoPoint)
toolbox.register("mutate", tools.mutFlipBit, indpb=0.1)
toolbox.register("select", tools.selTournament, tournsize=3)

```

- در این بخش، چارچوب Genetic Algorithm با کتابخانه DEAP تنظیم شد. ابتدا یک تابع شایستگی برای کمینه‌سازی تعریف گردید (FitnessMin با وزن -۱.۰) و سپس کلاس Individual ساخته شد تا هر فرد به صورت یک رشته باینری ۱/۰ (۱=انتخاب ویژگی، ۰=عدم انتخاب) نمایش داده شود. در ادامه، با toolbox ژن‌های تصادفی (attr_bool)، ساخت فرد با طول n_features (individual) و ساخت جمعیت (population) تعریف شدند و تابع ga_fitness_func به عنوان معیار ارزیابی برازندگی ثبت شد.
- برای تولید نسل‌های جدید، عملگر Two-Point Crossover (cxTwoPoint) و جهش Bit Flip Mutation (mutFlipBit با indpb=0.1) مطابق صورت سؤال به کار رفت. همچنین طبق صورت سؤال، روش انتخاب Tournament Selection با tournsize=3 استفاده شد که معمولاً در مسائل انتخاب ویژگی پایدارتر است. در مقایسه، روش Roulette Wheel نیز رایج است، اما چون انتخاب را به اختلاف‌های کوچک برازندگی حساس‌تر می‌کند، ممکن است در داده‌های نویزی باعث نوسان یا همگرایی زودرس شود.
- در اینجا مقدار tournsize=3 انتخاب شد تا یک تعادل مناسب بین فشار انتخاب و حفظ تنوع ایجاد شود؛ یعنی رقابت بین ۳ فرد باعث می‌شود افراد بهتر شانس بیشتری برای انتخاب شدن داشته باشند، اما فشار انتخاب آن قدر شدید نیست که تنوع جمعیت سریع از بین برود و الگوریتم دچار همگرایی زودرس شود.

```

population = toolbox.population(n=50)
# Evaluate initial population
for ind in population:
    ind.fitness.values = toolbox.evaluate(ind)

# Main GA loop (track convergence)
best_per_gen = []
avg_per_gen = []

for gen in range(50):
    # ----- Selection -----
    offspring = toolbox.select(population, len(population))
    offspring = list(map(toolbox.clone, offspring))

    # ----- Crossover -----
    for child1, child2 in zip(offspring[::2], offspring[1::2]):
        if random.random() < 0.9:
            toolbox.mate(child1, child2)

```



```
del child1.fitness.values
del child2.fitness.values

# ----- Mutation -----
for mutant in offspring:
    if random.random() < 0.1:
        toolbox.mutate(mutant)
        del mutant.fitness.values

# ----- Re-evaluation -----
invalid_ind = [ind for ind in offspring if not ind.fitness.valid]
for ind in invalid_ind:
    ind.fitness.values = toolbox.evaluate(ind)

# ----- Replacement -----
population[:] = offspring

# ----- Convergence stats -----
costs = [ind.fitness.values[0] for ind in population]
best_per_gen.append(min(costs))
avg_per_gen.append(float(np.mean(costs)))
# Best individual
best_ind = tools.selBest(population, 1)[0]

ga_mask = np.array(best_ind, dtype=bool)
ga_features = X.columns[ga_mask].to_list()
ga_selected_features = ga_features # keep name compatible with rest of
notebook

# Best GA cost
best_cost_ga = best_ind.fitness.values[0]

print("GA best cost:", best_cost_ga)
print("GA selected features:", int(ga_mask.sum()))
print("GA selected features:", ga_features)
GA best cost: 0.15085995085995085
GA selected features: 1
GA selected features: ['Credit_History']
```

- در این مرحله، اجرای کامل الگوریتم ژنتیک (GA) برای انتخاب ویژگی انجام شد. ابتدا جمعیت اولیه ارزیابی گردید و مقدار برازندگی هر فرد با `toolbox.evaluate` محاسبه و در `ind.fitness.values` ذخیره شد. سپس الگوریتم به مدت ۵۰ نسل اجرا شد.
- در هر نسل، ابتدا با Tournament Selection به اندازه‌ی کل جمعیت، افراد برگزیده انتخاب و با `clone` کپی شدند تا تغییرات روی نسخه‌ی جدید اعمال شود. بعد از آن، عملگر Two-Point Crossover با احتمال ۰.۹ روی جفت‌های متوالی اجرا شد و چون ساختار ژن‌ها تغییر می‌کند، مقدار برازندگی فرزندان حذف شد تا در مرحله‌ی بعد دوباره محاسبه شود. در ادامه، عملگر Bit Flip Mutation نیز با احتمال ۰.۱ روی هر فرد اعمال شد و همانند قبل، `fitness` افراد



- جهش یافته حذف گردید. سپس تنها افرادی که برازندگی معتبر نداشتند (`invalid_ind`) مجدد ارزیابی شدند تا از محاسبات اضافی جلوگیری شود. در پایان هر نسل، نسل جدید جایگزین جمعیت قبلی شد و برای بررسی روند همگرایی، بهترین هزینه و میانگین هزینه کل جمعیت در دو لیست `best_per_gen` و `avg_per_gen` ذخیره گردید.
- پس از پایان ۵۰ نسل، بهترین فرد نهایی با `tools.selBest` استخراج شد. این فرد به یک ماسک باینری (`ga_mask`) تبدیل شد تا ویژگی‌های انتخاب شده مشخص شوند و سپس نام ویژگی‌های متناظر از ستون‌های `X` استخراج و در `ga_features` ذخیره گردید.
 - ❖ نتیجه GA نشان داد بهترین فرد نهایی با `best cost = 0.15086` تنها یک ویژگی را انتخاب کرده است: `Credit_History`. این خروجی دقیقاً با هر دو اجرای PSO پیوسته و PSO باینری یکسان است (هم از نظر ویژگی منتخب و هم مقدار `cost`)، که یک پیام مهم دارد: انتخاب `Credit_History` یک نتیجه‌ی پایدار و مستقل از نوع الگوریتم جست‌وجو (PSO یا GA) است.
 - ❖ این همگرایی در این دیتاست، یعنی `Credit_History` به‌تنهایی بیشترین قدرت پیش‌بینی وضعیت وام را دارد و اضافه کردن سایر ویژگی‌ها یا بهبود معناداری در دقت ایجاد نمی‌کند، یا بهبودش آن‌قدر کم است که با توجه به تابع هزینه، ارزش افزایش تعداد ویژگی‌ها را ندارد. چون در تابع هزینه‌ی ما `feat_ratio` به‌صورت مستقیم جریمه‌ی انتخاب ویژگی‌های بیشتر را اعمال می‌کند و $\alpha=0.5$ هم تعادل بین دقت و تعداد ویژگی‌ها ایجاد می‌کند، هر دو الگوریتم به سمت راه‌حل «کم‌ویژگی ولی کارا» سوق داده شده‌اند و در نهایت به همان زیرمجموعه مینیمال رسیده‌اند.
 - ❖ در مقام مقایسه، PSO با حرکت ذرات و به‌روزرسانی بر اساس بهترین‌های فردی/جمعیتی جست‌وجو را پیمایش می‌کند، در حالی که GA با انتخاب، کراس‌اور و جهش نسل به نسل تکامل می‌دهد؛ با این وجود، یکسان شدن خروجی‌ها نشان می‌دهد فضای جواب احتمالاً یک مینیمم/جواب غالب و واضح دارد (یا حداقل در تنظیمات فعلی)، و اثر `Credit_History` آن‌قدر قوی است که هر دو روش به آن همگرا می‌شوند. بنابراین می‌توان نتیجه گرفت که برای این مسئله، تفاوت اصلی PSO و GA بیشتر در مکانیزم جست‌وجو و زمان اجرا است، نه در خروجی نهایی انتخاب ویژگی؛ چون هر دو به بهترین توازن بین «دقت» و «کمینه‌سازی ویژگی‌ها» رسیده‌اند.

د) ارزیابی عملکرد پس از انتخاب ویژگی و مقایسه PSO و GA

```
# -----
# (D) Evaluate RandomForest on selected features (Accuracy/Precision/Recall/F1)
# -----
rf_pso = make_rf(300, SEED)
rf_ga = make_rf(300, SEED)
rf_all = make_rf(300, SEED)
```



```

metrics_pso = evaluate_model(rf_pso, X_train, y_train, X_val, y_val,
                             feature_mask=pso_mask)
metrics_ga = evaluate_model(rf_ga, X_train, y_train, X_val, y_val,
                             feature_mask=ga_mask)
metrics_all = evaluate_model(rf_all, X_train, y_train, X_val, y_val,
                             feature_mask=None)

summary = pd.DataFrame([
    {"method": "All features", "n_features": int(n_features),
**{k: metrics_all[k] for k in ["accuracy", "precision", "recall", "f1"]}},
    {"method": "PSO", "n_features": int(pso_mask.sum()),
**{k: metrics_pso[k] for k in ["accuracy", "precision", "recall", "f1"]}},
    {"method": "GA", "n_features": int(ga_mask.sum()),
**{k: metrics_ga[k] for k in ["accuracy", "precision", "recall", "f1"]}},
]).sort_values("f1", ascending=False)

display(summary)

```

	method	n_features	accuracy	precision	recall	f1
1	PSO	1	0.789189	0.766234	0.975207	0.858182
2	GA	1	0.789189	0.766234	0.975207	0.858182
0	All features	11	0.756757	0.756757	0.925620	0.832714

```
=== Confusion matrices (Val) ===
```

```
All features:
```

```
[[ 28  36]
 [  9 112]]
```

```
PSO:
```

```
[[ 28  36]
 [  3 118]]
```

```
GA:
```

```
[[ 28  36]
 [  3 118]]
```

```
=== Classification report (best method on Val) ===
```

	precision	recall	f1-score	support
N	0.90	0.44	0.59	64
Y	0.77	0.98	0.86	121
accuracy			0.79	185
macro avg	0.83	0.71	0.72	185
weighted avg	0.81	0.79	0.77	185

- در این بخش، پس از انجام Feature Selection با دو روش PSO و GA، عملکرد مدل Random Forest (با n_estimators=300 برای ارزیابی پایدارتر) روی مجموعه‌ی اعتبارسنجی محاسبه و با



حالت استفاده از تمام ویژگی‌ها مقایسه شد. نتایج نشان می‌دهد هر دو الگوریتم PSO و GA دقیقاً یک زیرمجموعه‌ی یکسان انتخاب کرده‌اند (تنها Credit_History)، بنابراین معیارهای عملکرد آن‌ها نیز کاملاً برابر به دست آمده است.

❖ در مقایسه با حالت استفاده از همه ۱۱ ویژگی، انتخاب ویژگی باعث بهبود محسوس عملکرد شده است؛ به طوری که F1 از ۰.۸۳۳ به ۰.۸۵۸ و Accuracy از ۰.۷۵۷ به ۰.۷۸۹ افزایش یافته است. همچنین Recall از ۰.۹۲۶ به ۰.۹۷۵ رسیده که نشان می‌دهد مدل بعد از انتخاب ویژگی، نمونه‌های مثبت را بهتر شناسایی می‌کند و احتمال از دست دادن آن‌ها کمتر می‌شود؛ در عین حال Precision هم افت نکرده و کمی بهتر شده است.

❖ این بهبود در ماتریس درهم‌ریختگی هم کاملاً دیده می‌شود. در حالت All features در مقایسه با حالت PSO/GA تعداد False Negative‌ها از ۹ به ۳ کاهش یافته و همزمان True Positive‌ها از ۱۱۲ به ۱۱۸ افزایش پیدا کرده است؛ بنابراین مدل بعد از انتخاب ویژگی، وام‌های «قابل تایید» را کمتر اشتباه رد می‌کند. در مقابل، بخش مربوط به کلاس N تغییری نکرده (TN=28 و FP=36 ثابت مانده)، پس بهبود اصلی دقیقاً از سمت بهتر شدن تشخیص کلاس Y حاصل شده است.

❖ گزارش طبقه‌بندی روش برتر روی اعتبارسنجی نیز همین را تایید می‌کند: برای کلاس Y مقدار Recall≈0.98 و F1≈0.86 به دست آمده که نشان‌دهنده عملکرد بسیار خوب در شناسایی تاییدهاست. در مقابل، برای کلاس N مقدار Recall≈0.44 است؛ یعنی مدل هنوز بخشی از نمونه‌های N را به اشتباه Y پیش‌بینی می‌کند (FP‌های نسبتاً بالا). این الگو با نامتوازن بودن داده‌ها هم سازگار است (support: N=64 در برابر Y=121) و باعث می‌شود مدل به سمت پیش‌بینی Y تمایل بیشتری داشته باشد.

❖ از نظر سادگی مدل نیز نتیجه بسیار چشمگیر است: تعداد ویژگی‌ها از ۱۱ به ۱ کاهش یافته (حدود ۹۱٪ کمتر) و در عین حال عملکرد بهتر شده؛ این نشان می‌دهد Credit_History نقش غالب دارد و سایر ویژگی‌ها یا اطلاعات افزوده‌ی قابل توجهی نداشته‌اند یا اثرشان در برابر افزایش پیچیدگی کم بوده است. همچنین چون خروجی PSO و GA دقیقاً یکسان است (هم ویژگی منتخب و هم معیارها و ماتریس‌ها)، می‌توان نتیجه گرفت جواب به دست آمده پایدار و قابل اتکا بوده و مستقل از نوع الگوریتم به همان راه حل همگرا شده است.

ه) مقایسه اهمیت ویژگی‌ها و ویژگی‌های منتخب در PSO و GA

```
rf_all_full = make_rf(300, SEED)
rf_all_full.fit(X_train, y_train)

importances = pd.Series(rf_all_full.feature_importances_,
index=X_train.columns).sort_values(ascending=False)
```



```
print("Top 10 feature importances (RF on all features):")
display(importances.head(10))

print("\nPSO selected features vs top importances:")
if len(pso_features) > 0:
    display(importances.loc[pso_features].sort_values(ascending=False))
else:
    print("No features selected by PSO.")

print("\nGA selected features vs top importances:")
if len(ga_features) > 0:
    display(importances.loc[ga_features].sort_values(ascending=False))
else:
    print("No features selected by GA.")
```

Top 10 feature importances (RF on all features):

Credit_History	0.299555
Applicant_Income	0.178844
Loan_Amount	0.172105
Coapplicant_Income	0.112332
Area	0.068645
Dependents	0.048622
Married	0.029743
Education	0.028768
Term	0.024240
Gender	0.019996

dtype: float64

PSO selected features vs top importances:

Credit_History	0.299555
----------------	----------

dtype: float64

GA selected features vs top importances:

Credit_History	0.299555
----------------	----------

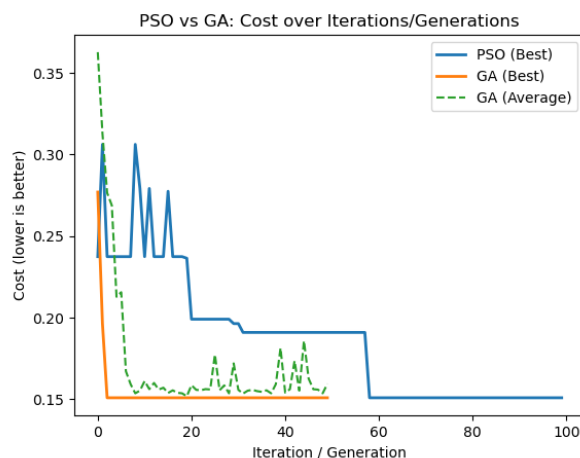
dtype: float64

- در این بخش ابتدا اهمیت ویژگی‌ها با آموزش یک Random Forest روی تمام ویژگی‌ها محاسبه شد که نشان داد Credit_History با مقدار حدود ۰.۳۰ مهم‌ترین ویژگی است و پس از آن به ترتیب Applicant_Income, Loan_Amount و Coapplicant_Income قرار دارند. سپس ویژگی‌های منتخب هر الگوریتم با این رتبه‌بندی مقایسه شد؛ نتیجه این بود که PSO و GA فقط ویژگی Credit_History را انتخاب کردند و این انتخاب با خروجی Feature Importance نیز کاملاً هم‌راستا است؛ یعنی این ویژگی به تنهایی بیشترین توان توضیح‌دهندگی تصمیم نهایی وام را در داده دارد. با این حال، در حالت کلی ممکن است انتخاب‌های PSO و GA با هم (یا با Feature Importance) متفاوت شوند، چون PSO/GA به دنبال بهترین زیرمجموعه بر اساس تابع برازندگی (ترکیب دقت و تعداد ویژگی‌ها) هستند، نه صرفاً «مهم‌ترین ویژگی منفرد». همچنین وجود همبستگی و هم‌پوشانی اطلاعات بین ویژگی‌ها، تصادفی بودن روند جست‌وجو و حساسیت به پارامترهایی مثل α ، نرخ



جهش/کراس اور یا حتی نوع تقسیم train/validation می تواند باعث شود الگوریتم ها به زیرمجموعه های متفاوت اما نزدیک به هم همگرا شوند.

و) مقایسه همگرایی PSO و GA بر اساس نمودار برازندگی (Cost) در طول تکرارها/نسل ها



- نمودار «PSO vs GA» نشان می دهد هر دو الگوریتم در نهایت به یک مقدار برازندگی نهایی یکسان (حدود ۰.۱۵۰۸۶) می رسند، بنابراین از نظر کیفیت جواب نهایی تفاوتی ندارند؛ اما از نظر سرعت همگرایی رفتارشان متفاوت است. منحنی GA (Best) خیلی سریع و در نسل های ابتدایی به کمینه می رسد و سپس ثابت می ماند، در حالی که PSO (Best) ابتدا نوسان دارد و بعد به صورت پله ای کاهش پیدا می کند تا نهایتاً در تکرارهای میانی/پایانی به همان مقدار کمینه برسد. پس از رسیدن به کمینه، منحنی PSO کاملاً تخت و پایدار می شود. در GA نیز بهترین مقدار ثابت است، اما منحنی GA (Average) پس از افت اولیه همچنان کمی نوسان دارد که طبیعی است، چون عملگرهای جهش و کراس اور باعث می شوند کیفیت افراد متوسط جمعیت تغییر کند حتی اگر بهترین فرد تغییر نکند. همچنین شکل پله ای و نوسانی PSO کاملاً طبیعی است؛ زیرا در PSO هر زمان یک ذره یا گروهی از ذرات به ناحیه بهتر برسند، مقدار best global ناگهان بهتر می شود و سپس تا یافتن بهبود بعدی ثابت می ماند، به همین دلیل منحنی حالت پله ای پیدا می کند و نوسان های ابتدای نمودار نیز ناشی از مرحله جست و جوی اولیه ذرات در فضای جواب است. جمع بندی اینکه GA سریع تر به جواب بهینه می رسد، PSO دیرتر اما پایدار همگرا می شود و هر دو از نظر دقت/برازندگی نهایی به نتیجه ای برابر و قابل اتکا رسیده اند.

پیش دو

(آ) بارگذاری داده ها و پیش پردازش اولیه و کاهش بعد توسط PCA



```
df = pd.read_csv("Mall_Customers.csv")
df.head(5)
```

	CustomerID	Gender	Age	Annual Income (k\$)	Spending Score (1-100)
0	1	Male	19	15	39
1	2	Male	21	15	81
2	3	Female	20	16	6
3	4	Female	23	16	77
4	5	Female	31	17	40

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 200 entries, 0 to 199
```

```
Data columns (total 5 columns):
```

#	Column	Non-Null Count	Dtype
0	CustomerID	200 non-null	int64
1	Gender	200 non-null	object
2	Age	200 non-null	int64
3	Annual Income (k\$)	200 non-null	int64
4	Spending Score (1-100)	200 non-null	int64

```
dtypes: int64(4), object(1)
```

```
memory usage: 7.9+ KB
```

```
df = df.dropna()
```

```
numeric_features = df.select_dtypes(include=["int64", "float64"])
```

```
print("Numeric features:", numeric_features.columns)
```

```
Numeric features: Index(['CustomerID', 'Age', 'Annual Income (k$)', 'Spending Score (1-100)'], dtype='object')
```

```
df=numeric_features.iloc[:,1:]
```

```
scaler = StandardScaler()
```

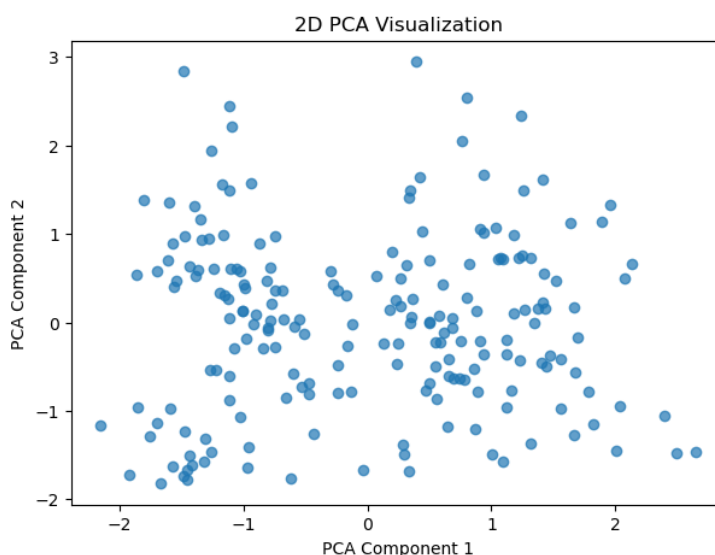
```
X_scaled = scaler.fit_transform(df)
```

- در این بخش، دیتاست *Mall_Customers* بارگذاری و بررسی شد. خروجی *info()* نشان داد داده شامل 200 رکورد و 5 ستون است و مقدار گم شده‌ای وجود ندارد؛ با این حال برای اطمینان، دستور *dropna()* اجرا شد تا در صورت وجود *NaN* حذف شوند. سپس فقط ویژگی‌های عددی انتخاب شدند که شامل *CustomerID*، *Age*، *Annual Income (k\$)* و *Spending Score (1-100)* هستند.
- از آنجا که *CustomerID* صرفاً شناسه است و نقش تحلیلی/خوشه‌بندی ندارد، از مجموعه ویژگی‌ها حذف شد و در نهایت ماتریس نهایی ویژگی‌ها با سه ستون *Age*، *Annual Income (k\$)* و *Spending Score (1-100)* تشکیل شد.



- در ادامه، طبق صورت سوال فقط روی ویژگی‌ها *StandardScaler* اعمال می‌شود تا مقیاس متغیرها یکسان شده و اثر اختلاف واحدها (مثلاً درآمد در برابر امتیاز خرید) حذف گردد.

```
pca = PCA(n_components=2, random_state=53)
X_pca = pca.fit_transform(X_scaled)
print('PCA explained variance ratio:', pca.explained_variance_ratio_)
print('Total explained variance (2D):', pca.explained_variance_ratio_.sum())
PCA explained variance ratio: [0.44266167 0.33308378]
Total explained variance (2D): 0.7757454566976746
```



- در این مرحله پس از استانداردسازی ویژگی‌ها، برای نمایش دوبعدی داده‌ها از *PCA* با ۲ مولفه استفاده شد.

❖ خروجی *PCA* نشان می‌دهد نسبت واریانس توضیح داده‌شده توسط دو مولفه‌ی اصلی به‌ترتیب ۰.۴۴۲۷ و ۰.۳۳۳۱ است و در مجموع حدود ۰.۷۷۵۷ ($\approx 77.6\%$) از تغییرات داده را در فضای دوبعدی پوشش می‌دهد. این مقدار نسبتاً بالا است و یعنی تصویر دوبعدی، نمای مناسبی از ساختار کلی داده ارائه می‌کند (هرچند بخشی از اطلاعات همچنان در مولفه‌های بعدی باقی می‌ماند).

❖ نمودار پراکندگی نیز توزیع نمونه‌ها را در فضای *PCA* نشان می‌دهد و می‌تواند برای دیدن الگوهای کلی، تراکم‌ها و جدایی نسبی گروه‌ها مفید باشد. با این حال، مطابق صورت سوال، این *PCA* صرفاً جهت بصری‌سازی استفاده شده و مدل‌های خوشه‌بندی باید در فضای اصلی ویژگی‌های استاندارد شده (*X_scaled*) آموزش داده شوند.

آ) الگوریتم KMeans



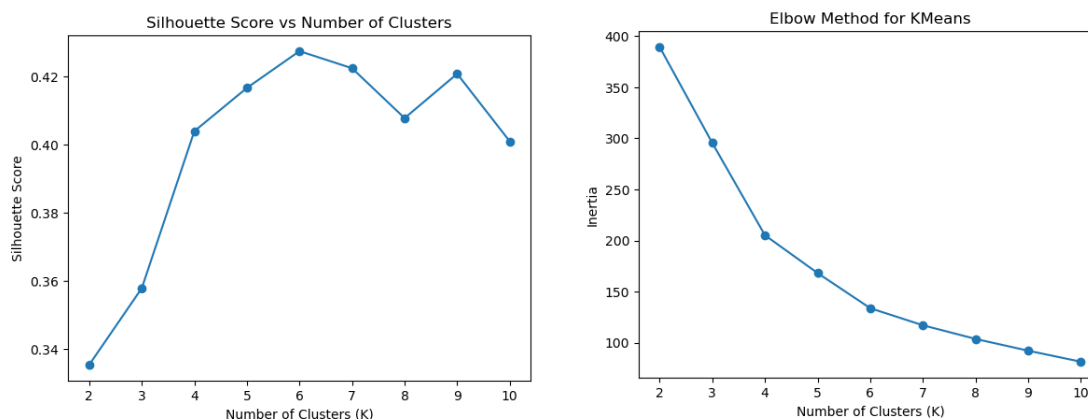
```
inertias = []
sil_scores = []
Ks = range(2, 11)

for k in Ks:
    km = KMeans(n_clusters=k, random_state=53, n_init=10)
    labels = km.fit_predict(X_scaled)
    inertias.append(km.inertia_)
    sil_scores.append(silhouette_score(X_scaled, labels))
results = pd.DataFrame({
    "K": list(Ks),
    "Inertia": inertias,
    "Silhouette": sil_scores
})
display(results)

best_k = int(results.loc[results["Silhouette"].idxmax(), "K"])
print(f"Best K by max silhouette: {best_k}")
```

		Inertia	Silhouette
K			
0	2	389.386189	0.335472
1	3	295.456245	0.357923
2	4	205.225147	0.403958
3	5	168.247580	0.416643
4	6	133.868334	0.427428
5	7	117.169115	0.422419
6	8	103.817972	0.407788
7	9	92.293165	0.420786
8	10	81.545784	0.400985

Best K by max silhouette: 6



- ❖ در این بخش، الگوریتم **KMeans** برای مقادیر $K \in \{2, \dots, 10\}$ روی داده‌های استاندارد شده (X_scaled) آموزش داده شد و برای هر K ، دو معیار **Silhouette** و **Inertia** گزارش گردید. طبق نتایج، مقدار **Inertia** با افزایش K به صورت یکنواخت کاهش یافته است (زیرا با زیاد شدن تعداد خوشه‌ها، خطای درون خوشه‌ای کمتر می‌شود)، اما این معیار به تنهایی برای انتخاب K کافی نیست چون همیشه با افزایش تعداد خوشه‌ها بهتر می‌شود.
- ❖ در مقابل، معیار **Silhouette** که همزمان فشردگی خوشه‌ها و جدایی بین خوشه‌ها را می‌سنجد، ابتدا افزایش پیدا کرده و در $K = 6$ به بیشترین مقدار خود یعنی 0.4274 رسیده است؛ سپس با افزایش بیشتر K روند آن کاهش/نوسان نشان می‌دهد (مثلاً در $K = 7$ برابر 0.4224 و در $K = 8$ برابر 0.4078).
- ❖ علاوه بر این، نمودار **Elbow Method** بر اساس **Inertia** نیز نشان می‌دهد کاهش **Inertia** تا حدود $K \approx 6$ نسبتاً تند است و بعد از آن شیب منحنی کمتر می‌شود؛ یعنی افزایش تعداد خوشه‌ها پس از این نقطه، بهبود چشمگیری در کاهش خطای درون خوشه‌ای ایجاد نمی‌کند. بنابراین، هم از دید بیشینه‌ی **Silhouette** و هم از دید نقطه آرنج (Elbow)، مقدار $K = 6$ به عنوان انتخاب نهایی یک گزینه‌ی منطقی و قابل دفاع است.

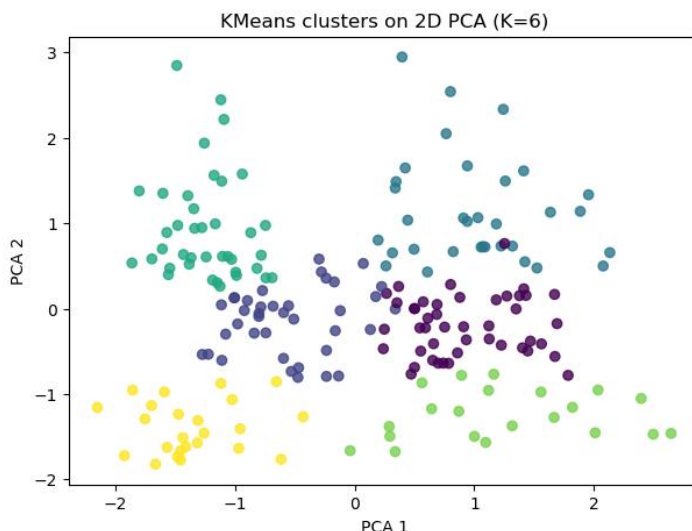
```
# Fit final KMeans using selected best_k (from silhouette)
kmeans_final = KMeans(n_clusters=best_k, random_state=53, n_init=10)
kmeans_labels = kmeans_final.fit_predict(X_scaled)

plt.figure(figsize=(7,5))
plt.scatter(X_pca[:,0], X_pca[:,1], c=kmeans_labels, alpha=0.8)
plt.xlabel("PCA 1")
plt.ylabel("PCA 2")
plt.title(f"KMeans clusters on 2D PCA (K={best_k})")
plt.show()

profile_kmeans = df.copy()
profile_kmeans["Cluster"] = kmeans_labels
```



```
display(profile_kmeans.groupby("Cluster").mean().round(2))
```



Cluster	Age	Annual Income (k\$)	Spending Score (1-100)
0	56.33	54.27	49.07
1	26.68	57.58	47.79
2	41.94	88.94	16.97
3	32.69	86.54	82.13
4	45.52	26.29	19.38
5	25.25	25.83	76.92

- در این مرحله، پس از انتخاب مقدار بهینه $K=6$ ، مدل $KMeans$ نهایی روی فضای اصلی ویژگی‌های استاندارد شده آموزش داده شد و برچسب خوشه‌ها استخراج گردید. سپس برای نمایش بصری، همین برچسب‌ها روی نگاشت PCA دوبعدی رسم شدند. نمودار نشان می‌دهد نقاط داده به ۶ گروه با جدایی قابل قبول تقسیم شده‌اند.

❖ برای تفسیر خوشه‌ها، میانگین ویژگی‌ها در هر خوشه (روی مقیاس اصلی) محاسبه شد که خلاصه آن به صورت زیر قابل برداشت است:



- *Cluster 0*: سن بالاتر (۵۶~) با درآمد متوسط (۵۴~) و امتیاز خرید متوسط (۴۹~) ⇒ مشتریان مسن تر با رفتار خرید معمولی.
 - *Cluster 1*: سن جوان (۲۷~) با درآمد متوسط رو به بالا (۵۸~) و امتیاز خرید متوسط (۴۸~) ⇒ جوان ها با خرید متعادل.
 - *Cluster 2*: سن میانسال (۴۲~) با درآمد بالا (۸۹~) و امتیاز خرید پایین (۱۷~) ⇒ درآمد بالا ولی کم خرج/کم خرید (محافظه کار).
 - *Cluster 3*: سن نسبتاً جوان (۳۳~) با درآمد بالا (۸۷~) و امتیاز خرید خیلی بالا (۸۲~) ⇒ مشتریان ارزشمند/هدف اصلی.
 - *Cluster 4*: سن میانسال (۴۶~) با درآمد پایین (۲۶~) و امتیاز خرید پایین (۱۹~) ⇒ کم درآمد و کم خرید.
 - *Cluster 5*: سن جوان (۲۵~) با درآمد پایین (۲۶~) اما امتیاز خرید بالا (۷۷~) ⇒ کم درآمد ولی پرخرج (احتمالاً خرید احساسی/تفریحی).
- ❖ این پروفایل سازی نشان می دهد *KMeans* توانسته گروه های معناداری از نظر سن، درآمد و الگوی هزینه کرد استخراج کند و به ویژه خوشه های (۳) و (۲) دو رفتار کاملاً متفاوت در درآمد بالا را جدا کرده اند (پرخرج در برابر کم خرج).

ج) الگوریتم *AgglomerativeClustering*

```
linkages = ["single", "complete", "average", "ward"]
agg_rows = []

K = best_k # use the same K chosen above for fair comparison

for link in linkages:
    agg = AgglomerativeClustering(n_clusters=K, linkage=link)
    labels = agg.fit_predict(X_scaled)
    sil = silhouette_score(X_scaled, labels)
    agg_rows.append({"Linkage": link, "K": K, "Silhouette": sil})

agg_results = pd.DataFrame(agg_rows).sort_values("Silhouette", ascending=False)
display(agg_results)

best_linkage = agg_results.iloc[0]["Linkage"]
print(f"Best linkage by silhouette: {best_linkage}")
```

	Linkage	K	Silhouette
3	ward	6	0.420117



2 average 6 0.389573

1 complete 6 0.374561

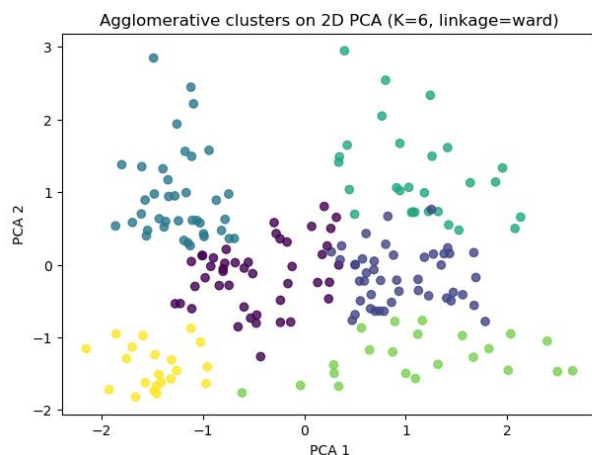
0 single 6 -0.042750

Best linkage by silhouette: ward

❖ در این بخش، برای مقایسه روش‌های مختلف **Agglomerative Clustering**، چهار نوع لینکیج **complete**، **average** و **ward** با تعداد خوشه ثابت $K = 6$ (برای مقایسه منصفانه با **KMeans**) روی داده‌های استاندارد شده اجرا شد و برای هر کدام **Silhouette Score** محاسبه گردید.

❖ از نظر مفهومی، تفاوت این لینکیج‌ها در معیار «فاصله بین دو خوشه» هنگام ادغام است: **single linkage** کمترین فاصله بین نقاط دو خوشه را ملاک می‌گیرد و معمولاً به پدیده *chaining* (زنجیره‌ای شدن خوشه‌ها) حساس است؛ **complete linkage** بیشترین فاصله بین نقاط دو خوشه را معیار قرار می‌دهد و خوشه‌های فشرده‌تر می‌سازد اما به نقاط دورافتاده حساس‌تر است؛ **average linkage** میانگین فاصله نقاط بین دو خوشه را در نظر می‌گیرد و معمولاً رفتاری متعادل‌تر دارد؛ و **ward linkage** به جای فاصله‌ی مستقیم، ادغامی را انتخاب می‌کند که کمترین افزایش در واریانس/خطای درون خوشه‌ای ایجاد شود، به همین دلیل معمولاً برای داده‌های عددی استاندارد شده خوشه‌های منسجم‌تری تولید می‌کند.

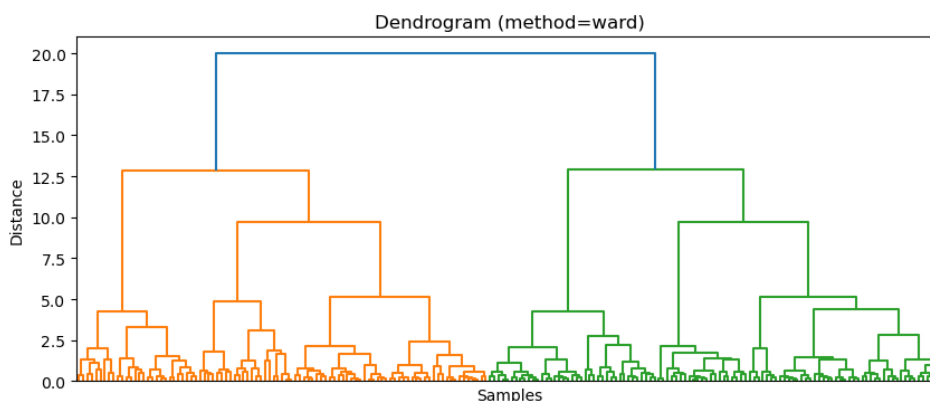
❖ نتایج سیلوئت نیز همین را تایید کرد: بهترین عملکرد مربوط به **ward** با مقدار $\text{Silhouette} = 0.4201$ است و پس از آن به ترتیب **average** (0.3896) و **complete** (0.3746) قرار می‌گیرند، در حالی که **single** مقدار منفی (-0.0428) تولید کرده که نشان‌دهنده خوشه‌بندی نامناسب و هم‌پوشانی زیاد خوشه‌هاست. بنابراین، بر اساس بیشینه‌سازی **Silhouette**، بهترین لینکیج **ward** گزارش می‌شود.





Cluster	Age	Annual Income (k\$)	Spending Score (1-100)
0	27.38	57.51	45.84
1	56.40	55.29	48.36
2	32.69	86.54	82.13
3	43.89	91.29	16.68
4	44.32	25.77	20.27
5	24.81	25.62	80.24

❖ در نهایت، با اجرای *Agglomerative* با لینکِج *ward* و $K = 6$ ، برچسب خوشه‌ها روی فضای *PCA* دوبعدی رسم شد. پروفایل میانگین خوشه‌ها نیز نشان می‌دهد الگوی خوشه‌ها از نظر سن، درآمد و امتیاز خرید مشابه تفسیر *KMeans* بوده و گروه‌های معناداری مانند «درآمد بالا-خرج بالا»، «درآمد بالا-خرج پایین»، و «درآمد پایین-خرج بالا/پایین» به خوبی تفکیک شده‌اند.



❖ این بخش برای نمایش ساختار خوشه‌بندی سلسله‌مراتبی، دندروگرام بر اساس ویژگی‌های استاندارد شده و با روش Ward رسم شد. دندروگرام نشان می‌دهد نمونه‌ها به صورت مرحله‌ای با افزایش «فاصله/عدم شباهت» به یکدیگر ادغام می‌شوند؛ هرچه ارتفاع اتصال‌ها (محور عمودی Distance) بیشتر باشد، یعنی ادغام دو خوشه در سطحی با اختلاف بیشتر انجام شده و در نتیجه آن خوشه‌ها از هم متمایزتر بوده‌اند. در این نمودار، مشاهده می‌شود در سطوح بالاتر یک جهش نسبتاً بزرگ در فاصله ادغام رخ می‌دهد (اتصال‌های مرتفع‌تر)، که بیانگر وجود چند گروه اصلی با جدایی قابل توجه است؛ بنابراین اگر یک خط برش افقی کمی پایین‌تر از این جهش در نظر گرفته شود، می‌توان به چند خوشه پایدار و معنادار رسید. به طور خاص، با توجه به انتخاب قبلی $K = 6$ و برتری لینکِج ward در سیلوئت، این



دندروگرام به صورت بصری نیز تایید می کند که تقسیم داده به چند خوشه محدود (از جمله حدود ۶ خوشه) قابل توجیه است و ساختار سلسله مراتبی داده از چنین برشی پشتیبانی می کند.

د) الگوریتم DBSCAN

```
eps_values = [0.2, 0.4, 0.6, 0.8, 1.0]
min_samples_values = [3, 5, 10]

results = []

for eps in eps_values:
    for ms in min_samples_values:
        db = DBSCAN(eps=eps, min_samples=ms)
        labels = db.fit_predict(X_scaled)

        # Identify noise
        noise_ratio = np.sum(labels == -1) / len(labels)

        # Number of clusters (exclude noise)
        n_clusters = len(set(labels)) - (1 if -1 in labels else 0)

        # Silhouette only on non-noise points
        if n_clusters > 1 and np.sum(labels != -1) > 1:
            sil = silhouette_score(
                X_scaled[labels != -1],
                labels[labels != -1]
            )
        else:
            sil = np.nan

        results.append({
            "eps": eps,
            "min_samples": ms,
            "Clusters": n_clusters,
            "Noise Ratio": noise_ratio,
            "Silhouette": sil
        })

results_df = pd.DataFrame(results)
display(results_df)

# Pick best DBSCAN setting by highest silhouette (ignoring NaN)
best_db = results_df.dropna(subset=["Silhouette"]).sort_values("Silhouette",
    ascending=False).head(1)
if len(best_db) == 0:
    print("No DBSCAN setting produced >1 cluster with enough non-noise points
    for silhouette.")
else:
    best_eps = float(best_db.iloc[0]["eps"])
    best_ms = int(best_db.iloc[0]["min_samples"])
    print("Best DBSCAN by silhouette:")
```



display(best_db)					
	eps	min_samples	Clusters	Noise Ratio	Silhouette
0	0.2	3	11	0.805	0.645880
1	0.2	5	1	0.975	NaN
2	0.2	10	0	1.000	NaN
3	0.4	3	10	0.295	0.442575
4	0.4	5	6	0.490	0.519023
5	0.4	10	2	0.850	0.766073
6	0.6	3	3	0.070	0.214881
7	0.6	5	2	0.140	0.273047
8	0.6	10	4	0.330	0.529589
9	0.8	3	1	0.015	NaN
10	0.8	5	1	0.030	NaN
11	0.8	10	1	0.115	NaN
12	1.0	3	1	0.005	NaN
13	1.0	5	1	0.010	NaN
14	1.0	10	1	0.025	NaN
Best DBSCAN by silhouette:					
	eps	min_samples	Clusters	Noise Ratio	Silhouette
5	0.4	10	2	0.85	0.766073

- در این بخش، الگوریتم **DBSCAN** با یک شبکه کوچک از پارامترها مطابق صورت سوال اجرا شد؛ به این صورت که $\epsilon \in \{0.2, 0.4, 0.6, 0.8, 1.0\}$ و $\min_samples \in \{3, 5, 10\}$ روی داده‌های استاندارد شده امتحان شدند. برای هر تنظیم، سه شاخص گزارش شد: **تعداد خوشه‌ها** (بدون در نظر گرفتن نویز)، **نسبت نویز** (درصد برچسب‌های -۱) و **Silhouette** که فقط روی نقاط غیرنویز محاسبه شد (چون وجود نویز می‌تواند محاسبه سیلوئت را منحرف کند).

❖ نتایج نشان می‌دهد برخی تنظیم‌ها عملاً به یک خوشه (یا هیچ خوشه) منجر شده‌اند و بنابراین سیلوئت قابل محاسبه نیست. به طور کلی، وقتی ϵ خیلی بزرگ باشد (۰.۸ و ۱.۰)، اغلب نقاط در یک خوشه ادغام



می‌شوند و DBSCAN توان تفکیک خوشه‌ها را از دست می‌دهد؛ و وقتی ϵ خیلی کوچک باشد (مثلاً ۰.۲)، بسیاری از نقاط به عنوان نویز شناخته می‌شوند و نسبت نویز بسیار بالا می‌رود.

❖ بر اساس بیشینه‌سازی Silhouette (با حذف حالت‌های NaN)، بهترین تنظیم به دست آمده عبارت است از:

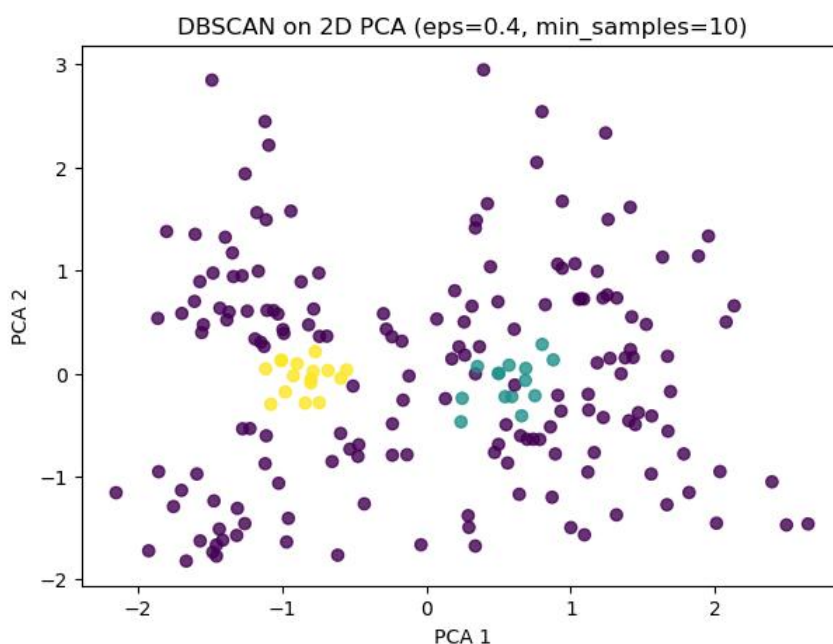
○ $\min_samples = 10$ و $\epsilon = 0.4$

○ تعداد خوشه‌ها = ۲

○ نسبت نویز = ۰.۸۵

○ Silhouette (روی نقاط غیرنویز) = ۰.۷۶۶۱

❖ این مقدار سیلوئت بالا نشان می‌دهد دو خوشه تشکیل شده روی نقاط غیرنویز، از نظر جدایی و انسجام کیفیت خوبی دارند؛ اما باید توجه کرد که نسبت نویز بسیار زیاد (۸۵٪) است، یعنی DBSCAN با این تنظیم بخش عمده‌ای از داده‌ها را نویز در نظر گرفته و فقط یک زیرمجموعه کوچک را خوشه‌بندی کرده است. بنابراین، اگر هدف صرفاً بیشینه کردن سیلوئت طبق صورت سؤال باشد، این تنظیم بهترین است؛ اما از نظر کاربردی، ممکن است تنظیم‌هایی مانند $\epsilon = 0.4, \min_samples = 5$ (با ۶ خوشه و نویز ۴۹٪) یا $\epsilon = 0.6, \min_samples = 10$ (با ۴ خوشه و نویز ۳۳٪) توازن بهتری بین «پوشش داده» و «کیفیت خوشه‌بندی» ارائه دهند.





Cluster	Age	Annual Income (k\$)	Spending Score (1-100)
0	48.14	57.57	46.71
1	22.06	60.56	50.12

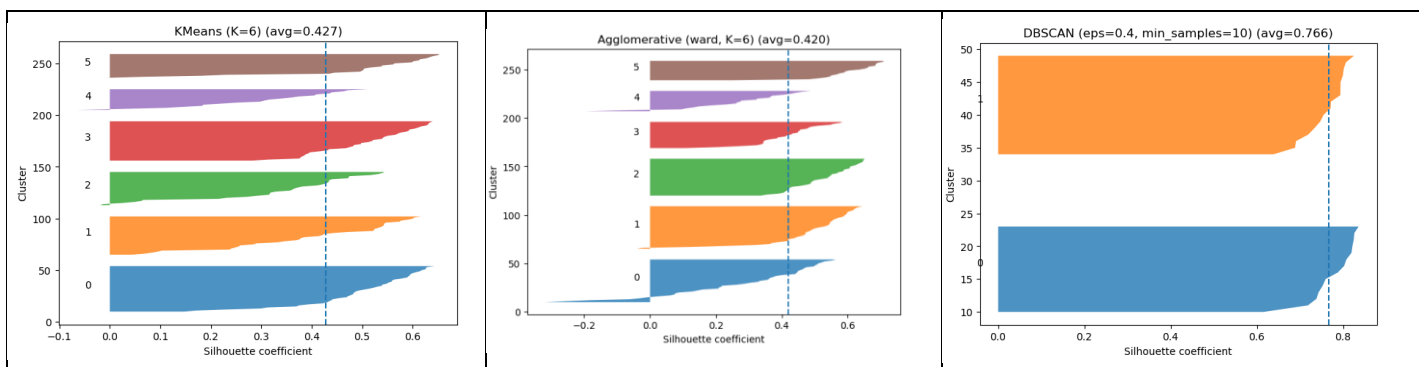
❖ با تنظیم منتخب DBSCAN یعنی $\epsilon = 0.4$ و $\min_samples = 10$ خروجی خوشه‌بندی نشان

می‌دهد مدل تنها ۲ خوشه شناسایی کرده و بخش عمده‌ای از داده‌ها را به‌عنوان نویز (برچسب -۱) طبقه‌بندی نموده است. این رفتار با مقدار $Noise\ Ratio = 0.85$ سازگار است؛ یعنی حدود ۸۵٪ نمونه‌ها خارج از نواحی با چگالی کافی تشخیص داده شده‌اند. در نمودار نمایش روی PCA دوبعدی نیز دو خوشه‌ی کوچک و فشرده قابل مشاهده است و سایر نقاط به‌صورت پراکنده در فضای ویژگی‌ها قرار گرفته‌اند که به‌عنوان نویز نمایش داده می‌شوند.

❖ از منظر تفسیر خوشه‌ها (روی نقاط غیرنویز)، میانگین ویژگی‌ها نشان می‌دهد: خوشه ۰ عمدتاً شامل افراد با سن بالاتر (حدود ۴۸) و درآمد متوسط (حدود ۵۷.۶) با امتیاز خرید متوسط (حدود ۴۶.۷) است؛ در حالی که خوشه ۱ شامل افراد جوان‌تر (حدود ۲۲) با درآمد نسبتاً بالاتر (حدود ۶۰.۶) و امتیاز خرید نزدیک به متوسط (حدود ۵۰.۱) می‌باشد.

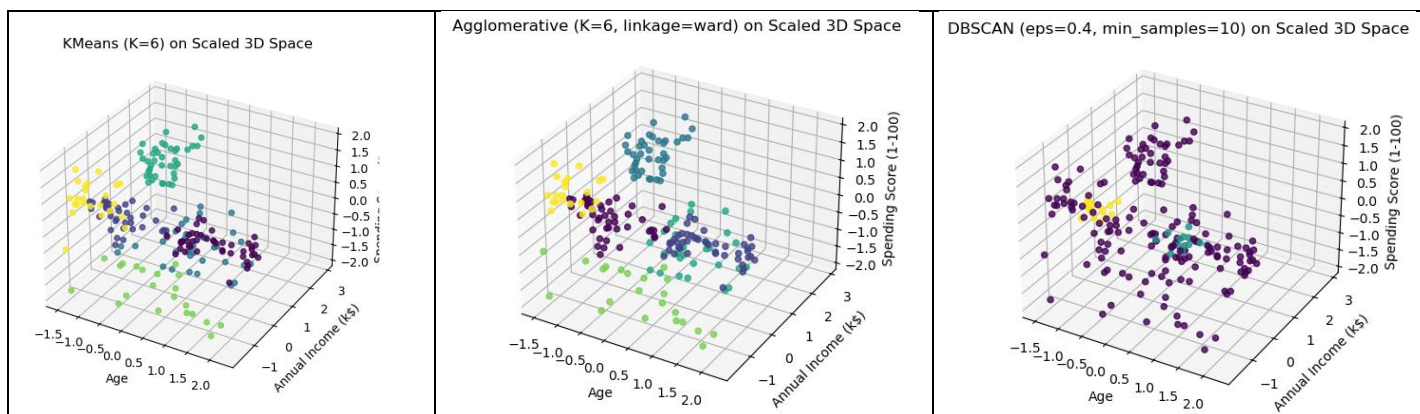
❖ در مجموع، اگرچه این تنظیم به دلیل Silhouette بالا روی نقاط غیرنویز کیفیت جدایی مناسبی برای خوشه‌های استخراج‌شده دارد، اما به دلیل نسبت نویز بسیار زیاد، DBSCAN در این حالت بیشتر نقش شناسایی «هسته‌های بسیار متراکم» را ایفا کرده و بخش بزرگی از مشتریان را خارج از خوشه‌ها قرار داده است؛ بنابراین برای خوشه‌بندی کامل تمام داده‌ها، ممکن است نیاز به تنظیمی با توازن بهتر بین کیفیت خوشه‌ها و پوشش داده وجود داشته باشد.

ه) مقایسه الگوریتم‌های KMEANS و Agglomerative Clustering و DBSCAN





- ❖ این سه نمودار سیلوئت از نظر رفتاری کاملاً قابل انتظار هستند و دقیقاً نشان می‌دهند هر الگوریتم چه نوع خوشه‌هایی تولید کرده است. در $KMeans (K=6)$ میانگین سیلوئت حدود 0.427 است و بیشتر نوارهای هر خوشه در ناحیه مثبت قرار دارند؛ یعنی اکثر نقاط از خوشه خودشان به خوبی حمایت می‌کنند، هرچند وجود بخش‌هایی نزدیک صفر نشان می‌دهد تعدادی از نقاط مرزی هستند و جداسازی همه خوشه‌ها صددرصد ایده‌آل نیست.
- ❖ در Agglomerative با لینکیج ward و $K=6$ میانگین سیلوئت کمی پایین‌تر (0.420) است و نسبت به $KMeans$ تعدادی مقدار نزدیک صفر و حتی کمی منفی بیشتر دیده می‌شود؛ این یعنی در برخی خوشه‌ها، تعدادی نمونه به خوشه دیگری نزدیک‌ترند و هم‌پوشانی/عدم قطعیت خوشه‌ای اندکی بیشتر از $KMeans$ است، با این حال کیفیت کلی هنوز نزدیک و قابل قبول است.
- ❖ اما در $DBSCAN (eps=0.4, min_samples=10)$ میانگین سیلوئت 0.766 خیلی بالاتر است؛ با این تفاوت مهم که در کد شما برای $DBSCAN$ ، سیلوئت فقط روی نقاط غیرنویز محاسبه شده است. چون این تنظیم نسبت نویز بسیار بالا دارد، $DBSCAN$ عملاً فقط «هسته‌های بسیار متراکم» را نگه داشته و بقیه نقاط را حذف (نویز) کرده است؛ بنابراین سیلوئت بالا بیشتر نشان‌دهنده جدایی بسیار خوب همان تعداد کم نقاط باقی‌مانده است، نه اینکه کل دیتاست بهتر خوشه‌بندی شده باشد. به بیان دقیق‌تر، $KMeans$ و ward تقریباً کل داده را به 6 بخش تقسیم می‌کنند و سیلوئت متوسط‌تری می‌گیرند، در حالی که $DBSCAN$ با حذف بخش بزرگی از داده، روی نقاط باقی‌مانده دو خوشه بسیار واضح می‌سازد و طبیعی است که سیلوئت بالاتری ثبت شود.
- ❖ در نتیجه اگر هدف «خوشه‌بندی کامل مشتری‌ها» باشد، $KMeans$ و ward مناسب‌ترند چون تقریباً همه نقاط را پوشش می‌دهند و سیلوئت متوسط ولی پایدار دارند. اگر هدف «استخراج چند گروه خیلی متراکم و متمایز» باشد، $DBSCAN$ به‌ظاهر بهترین سیلوئت را می‌دهد، اما باید همراه آن حتماً ذکر شود که این کیفیت بالا با نویز بسیار زیاد و پوشش پایین داده به‌دست آمده است.



- ❖ این بخش شامل نمایش سه بعدی نتایج خوشه بندی در فضای اصلی استاندارد شده (سه ویژگی Age، Spending Score، Annual Income) است تا ساختار خوشه ها بدون اتکا به PCA و صرفاً بر اساس ویژگی های واقعی داده مشاهده شود.
- ❖ در نمودار KMeans با $K = 6$ خوشه ها عمدتاً به صورت گروه های نسبتاً فشرده و قابل تفکیک دیده می شوند و جداسازی آن ها بیشتر تحت تأثیر ترکیب «درآمد سالانه» و «امتیاز خرید» شکل گرفته است؛ با این حال در نواحی مرکزی، هم پوشانی محدودی بین برخی خوشه ها مشاهده می شود که با مقدار سیلوئت متوسط این روش سازگار است.
- ❖ در Agglomerative با لینکیج ward و $K = 6$ الگوی کلی خوشه ها مشابه KMeans است، اما در برخی نواحی خوشه ها شکل نامتقارن تر یا کشیده تری دارند و تعداد نقاط مرزی کمی بیشتر دیده می شود؛ این تفاوت ظریف می تواند علت کاهش اندک سیلوئت نسبت به KMeans را توضیح دهد.
- ❖ در مقابل، نمودار DBSCAN با $\epsilon = 0.4$ و $\text{min_samples} = 10$ نشان می دهد تنها نواحی بسیار متراکم به عنوان خوشه شناسایی شده اند و بخش قابل توجهی از داده ها خارج از این نواحی قرار گرفته و عملاً به عنوان نویز در نظر گرفته می شوند. بنابراین، هرچند DBSCAN از نظر سیلوئت روی نقاط غیرنویز مقدار بالاتری دارد، نمایش سه بعدی تایید می کند که این مقدار بالا ناشی از خوشه بندی «هسته های کوچک و بسیار متراکم» است و لزوماً به معنای خوشه بندی کامل کل داده نیست.
- ❖ در مجموع، تصاویر سه بعدی نشان می دهند KMeans و Agglomerative برای هدف «تقسیم بندی جامع مشتریان» مناسب تر هستند، در حالی که DBSCAN بیشتر برای استخراج خوشه های بسیار متراکم و شناسایی نقاط پراکنده/نویز کاربرد دارد.



پیش سه

(آ) معادله بهروزرسانی Q-learning و نقش α ، γ و greedy ϵ - و چرایی off-policy بودن

معادله‌ی بهروزرسانی Q -learning (برای زوج حالت-عمل (s_t, a_t)) معمولاً به یکی از دو فرم زیر نوشته می‌شود که معادلند:

$$Q_{t+1}(s_t, a_t) = (1 - \alpha) Q_t(s_t, a_t) + \alpha (r_{t+1} + \gamma \max_a Q_t(s_{t+1}, a))$$

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

در این روابط، α نرخ یادگیری است و وزن «اطلاعات جدید» را نسبت به مقدار قبلی Q تعیین می‌کند: اگر α بزرگ باشد Q سریع‌تر تغییر می‌کند (یادگیری سریع‌تر) ولی ممکن است نوسان و بی‌ثباتی بیشتر شود؛ اگر α کوچک باشد تغییرات آرام‌تر و پایدارتر است اما همگرایی کندتر می‌شود.

γ ضریب تنزیل است و مشخص می‌کند پاداش‌های آینده چقدر مهم باشند. γ نزدیک ۰ یعنی عامل بیشتر به پاداش فوری توجه می‌کند، و γ نزدیک ۱ یعنی پاداش‌های بلندمدت هم نقش پررنگی دارند.

عبارت $\max_a Q(s_{t+1}, a)$ هم «بهترین ارزش قابل انتظار در حالت بعدی» را نشان می‌دهد و در واقع هدف (target) یادگیری را می‌سازد.

برای انتخاب عمل در عمل کردن (نه در بهروزرسانی)، معمولاً از سیاست ϵ - greedy استفاده می‌شود تا تعادل بین اکتشاف و بهره‌برداری برقرار شود: با احتمال ϵ یک عمل تصادفی انتخاب می‌کنیم (برای کشف گزینه‌های جدید) و با احتمال $1 - \epsilon$ بهترین عمل فعلی را بر اساس Q انتخاب می‌کنیم (برای استفاده از دانسته‌ها):

$$a = \begin{cases} \text{random action} & \text{with prob } \epsilon \\ \arg \max_a Q(s, a) & \text{with prob } 1 - \epsilon \end{cases}$$

در نهایت، Q -learning یک روش *off-policy* محسوب می‌شود چون سیاستی که با آن داده جمع‌آوری می‌شود (*behavior policy*) مثل ϵ - greedy می‌کند با سیاستی که در بهروزرسانی «هدف‌گذاری» می‌شود (*target policy*) یکسان نیست.

به طور مشخص، در فرمول Q -learning هدف از عبارت $\max_a Q(s_{t+1}, a)$ می‌آید که معادل فرض انتخاب حریصانه‌ترین عمل در حالت بعدی است، حتی اگر در عمل به خاطر ϵ - greedy یک عمل غیربهبینه انتخاب



شده باشد. بنابراین یادگیری در حال برآورد ارزش سیاست بهینه (حریصانه) است، در حالی که رفتار واقعی ممکن است تصادفی تر باشد؛ همین جدایی رفتار و هدف، دلیل *off-policy* بودن *Q-learning* است.

ب) محاسبه مقدار جدید $Q(s_0, a_1)$ با یک گام به روزرسانی *Q-learning*

با استفاده از به روزرسانی *Q-learning*

$$Q_{t+1}(s_t, a_t) = (1 - \alpha) Q_t(s_t, a_t) + \alpha (r_t + \gamma \max_a Q_t(s_{t+1}, a))$$

اطلاعات مسئله:

$$Q_t(s_0, a_1) = 0 \quad (\text{چون جدول } Q \text{ ابتدا صفر است})$$

$$r_t = +2$$

$$\gamma = 0.9$$

$$\alpha = 0.2$$

$$\max_a Q_t(s_1, a) = 1.5$$

۱. محاسبه‌ی هدف: *(Target)*

$$r_t + \gamma \max_a Q_t(s_{t+1}, a) = 2 + 0.9 \times 1.5 = 2 + 1.35 = 3.35$$

۲. به روزرسانی مقدار: Q

$$Q_{t+1}(s_0, a_1) = (1 - 0.2) \times 0 + 0.2 \times 3.35 = 0 + 0.67 = 0.67$$

ج) دو عامل موثر بر کندی همگرایی/ناپایداری در *Q-learning* و راهکار عملی

۱. پاداش‌دهی نامناسب یا مقیاس پاداش خیلی بزرگ/کوچک

اگر پاداش‌ها خیلی بزرگ، نویزی یا با مقیاس نامتوازن باشند، مقدار Q نوسان می‌کند و یادگیری ناپایدار می‌شود (یا اگر خیلی کوچک باشند، یادگیری کند می‌شود).

راهکار عملی: نرمال‌سازی/مقیاس‌دهی پاداش یا کلیپ کردن پاداش (مثلاً محدود کردن پاداش به بازه $[-1, 1]$) تا آپدیت‌ها کنترل‌شده و پایدار شوند.

۲. انتخاب نامناسب ϵ در *greedy* - ϵ (اکتشاف کم یا زیاد)

اگر ϵ خیلی کم باشد، عامل زود وارد بهره‌برداری می‌شود و ممکن است در سیاست‌های ضعیف گیر کند (همگرایی)



کند/بد). اگر ϵ خیلی زیاد و ثابت بماند، رفتار بیش از حد تصادفی می شود و Q به سختی تثبیت می گردد.
راهکار عملی کاهش تدریجی ϵ (مثلا از ۱ شروع و به ۰.۰۵ برسد) تا ابتدا اکتشاف کافی انجام شود و سپس یادگیری پایدار و همگرا گردد.